



Annexe 3 — Patrons utiles aux architectures distribuées

Année 2026–2027 Enseignant Bachar Rima

À QUOI SERT CETTE ANNEXE

Le CM 01 (section 4) identifie **cinq patrons** qui constituent l’ossature de tous les intergiciels étudiés dans cette UE. Cette annexe les présente un par un, à partir d’un **problème concret**, avec un exemple complet et exécutable, puis montre où le même principe réapparaît dans une architecture distribuée. C’est un document de référence, à consulter quand un mécanisme de RMI ou de Spring vous paraît arbitraire.

PÉRIMÈTRE

Chaque patron ci-dessous dispose d’un exemple compilable dans [resources/design-patterns/](#), avec `javac` seul, sans aucune dépendance. Les listings et les sorties imprimés ici sont *inclus directement* depuis ces fichiers : ce que vous lisez est exactement ce que vous exécuterez.

1. Vue d’ensemble

PATRON	QUESTION À LAQUELLE IL RÉPOND	OÙ VOUS LE RENCONTREZ
Proxy	Comment appeler un objet qui n’est pas dans mon espace d’adressage ?	le stub RMI · le client gRPC
Adapter	Comment réutiliser un service existant dont l’interface ne convient pas ?	adaptateurs de protocole · passerelles
Factory	Qui crée les objets distants, et selon quels paramètres ?	<code>RMIClientSocketFactory</code> · fabriques de canaux
Interceptor	Comment ajouter journalisation, sécurité ou mesure sans toucher au code ?	chaînes de <i>handlers</i> SOAP · intercepteurs gRPC · AOP de Spring
Observer	Comment prévenir plusieurs modules d’un changement d’état ?	l’étape A6 du TP · notifications asynchrones

PATRON, PRINCIPE, CANEVAS

Un **principe** de conception est une règle de haut niveau, indépendante du langage (les principes SOLID, par exemple). Un **patron** est un schéma de solution réutilisable pour un problème récurrent. Un **canevas** (*framework*) est du code réutilisable qui met en œuvre plusieurs patrons — et qui, contrairement à une bibliothèque, *appelle* votre code au lieu d’être appelé par lui.

1.1. Comment lire chaque fiche

Les cinq fiches suivent le même fil :

1. **le problème**, et pourquoi la solution naïve ne tient pas ;
2. **l’intention** du patron, ses participants et sa structure ;
3. **l’exemple**, fichier par fichier, puis son exécution réelle ;
4. ce que le patron **permet de faire évoluer** ;

5. **du patron objet à l'intergiciel** : ce que vaut le même principe dans une architecture distribuée, et où vous le retrouvez dans l'UE ;
6. les **pièges** les plus fréquents.

UN PATRON N'EST PAS UN MÉCANISME DISTRIBUÉ

Les exemples de cette annexe s'exécutent **dans une seule JVM**. Un proxy local n'est pas un stub RMI ; un observateur local n'est pas un rappel à travers le réseau. Chaque fiche sépare donc explicitement le patron *en conception objet* de la façon dont la *même idée structurelle* est exploitée par un intergiciel — avec ce que la distribution ajoute : latence, pannes partielles, sérialisation, exportation.

COMPILER ET EXÉCUTER UN EXEMPLE

Depuis `resources/design-patterns/<patron>` (JDK 17 ou plus) :

```
$ javac -d out $(find src -name "*.java")
$ java -cp out patterns.<patron>.Demo<Patron>
```

Sous Windows (PowerShell) : `javac -d out (Get-ChildItem -Recurse -Filter *.java src).FullName`.
Chaque dossier contient un `README.md` et le fichier `expected-output.txt` reproduit dans cette annexe.

2. Proxy — un représentant qui contrôle l'accès

2.1. Le problème

Un tableau de bord météo interroge un service de prévision. Chaque appel déclenche un calcul coûteux — plusieurs secondes dans la réalité. Le tableau de bord demande souvent la même ville deux fois de suite.

On voudrait **éviter les calculs inutiles** sans toucher ni au tableau de bord, ni au service. Les solutions naïves sont toutes insatisfaisantes :

- ajouter un cache *dans le tableau de bord* : chaque client devra le réécrire ;
- ajouter un cache *dans le service* : on mélange le calcul et la politique d'accès, et l'on ne peut plus avoir le service sans cache ;
- faire appeler au client une autre méthode, `previsionAvecCache` : le client doit savoir qu'un cache existe.

La partie 1 de l'exemple montre la situation de départ : trois appels, trois calculs, dont un inutile.

2.2. Intention et participants

INTENTION

Fournir un **substitut** d'un objet, qui présente **la même interface** et **contrôle l'accès** à l'objet réel. Le client ne sait pas s'il parle à l'objet ou à son représentant.

RÔLE	DANS L'EXEMPLE	RESPONSABILITÉ
Subject	<code>ServiceMeteo</code>	le contrat commun, seul type connu du client
RealSubject	<code>ServiceMeteoReel</code>	fait le vrai travail
Proxy	<code>ProxyCacheMeteo</code> , puis <code>StubSimule</code>	même contrat ; détient une référence vers l'objet réel et décide <i>quand</i> et <i>comment</i> l'appeler
Client	<code>tableauDeBord(...)</code>	ne dépend que de <code>ServiceMeteo</code>

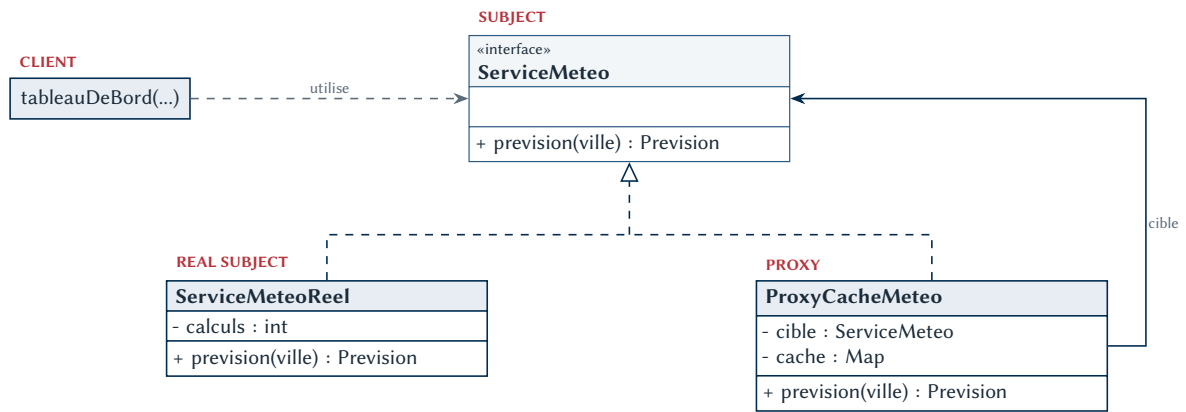


FIGURE 1 – Structure du patron Proxy dans l'exemple

2.3. L'exemple, fichier par fichier

Le contrat, la donnée renvoyée et l'objet réel :

Listing 1 – ServiceMeteo.java

```

1 package patterns.proxy;
2
3 /** SUBJECT : le contrat que connait le client. */
4 public interface ServiceMeteo {
5
6     Prevision prevision(String ville);
7 }
  
```

Listing 2 – Prevision.java

```

1 package patterns.proxy;
2
3 import java.io.Serializable;
4
5 /** Donnee renvoyee par le service. Serializable : utile a la partie 3. */
6 public record Prevision(String ville, int celsius, String ciel) implements Serializable { }
  
```

Listing 3 – ServiceMeteoReel.java

```
1 package patterns.proxy;
2
3 /**
4  * REAL SUBJECT : l'objet qui fait le vrai travail.
5  * Chaque appel est couteux (on imagine un calcul de plusieurs secondes).
6  */
7 public class ServiceMeteoReel implements ServiceMeteo {
8
9     private int calculs;
10
11     @Override
12     public Prevision prevision(String ville) {
13         calculs++;
14         System.out.println("    [service reel] calcul couteux pour " + ville);
15         int t = 10 + Math.abs(ville.hashCode() % 20);
16         return new Prevision(ville, t, t > 20 ? "ensoleille" : "nuageux");
17     }
18
19     public int calculs() { return calculs; }
20 }
```

Le proxy écrit à la main. Il **implémente la même interface** et **délègue** à l'objet réel; entre les deux, il décide :

Listing 4 – ProxyCacheMeteo.java

```
1 package patterns.proxy;
2
3 import java.util.HashMap;
4 import java.util.Map;
5
6 /**
7  * PROXY ecrit a la main : meme interface que le service reel, et une
8  * reference vers lui. Il CONTROLE l'accès : ici, il evite de recalculer
9  * une prevision deja connue.
10 */
11 public class ProxyCacheMeteo implements ServiceMeteo {
12
13     private final ServiceMeteo cible;
14     private final Map<String, Prevision> cache = new HashMap<>();
15
16     public ProxyCacheMeteo(ServiceMeteo cible) {
17         this.cible = cible;
18     }
19
20     @Override
21     public Prevision prevision(String ville) {
22         Prevision p = cache.get(ville);
23         if (p != null) {
24             System.out.println("    [proxy] " + ville + " servie depuis le cache");
25             return p;
26         }
27         p = cible.prevision(ville); // delegation a l'objet reel
28         cache.put(ville, p);
29         return p;
30     }
31 }
```

Un second proxy, **engendré à l'exécution**. Il ne sait rien de la météo : il transforme n'importe quel appel en une *Requete*, la sérialise, et confie les octets à un « squelette » qui fait l'opération inverse. C'est une **simulation**, dans une seule JVM, du principe d'un stub.

Listing 5 – Requete.java

```
1 package patterns.proxy;
2
3 import java.io.Serializable;
4
5 /** Description d'un appel, transformable en octets : nom, types, arguments. */
6 public record Requete(String methode, Class<?>[] types, Object[] arguments)
7     implements Serializable { }
```

Listing 6 – Octets.java

```
1 package patterns.proxy;
2
3 import java.io.ByteArrayInputStream;
4 import java.io.ByteArrayOutputStream;
5 import java.io.IOException;
6 import java.io.ObjectInputStream;
7 import java.io.ObjectOutputStream;
8
9 /** Serialisation Java native : objet <-> flux binaire. */
10 final class Octets {
11
12     private Octets() { }
13
14     static byte[] versOctets(Object o) throws IOException {
15         ByteArrayOutputStream buffer = new ByteArrayOutputStream();
16         try (ObjectOutputStream out = new ObjectOutputStream(buffer)) {
17             out.writeObject(o);
18         }
19         return buffer.toByteArray();
20     }
21
22     static Object depuisOctets(byte[] octets) throws IOException, ClassNotFoundException {
23         try (ObjectInputStream in =
24             new ObjectInputStream(new ByteArrayInputStream(octets))) {
25             return in.readObject();
26         }
27     }
28 }
```

Listing 7 – StubSimule.java

```
1 package patterns.proxy;
2
3 import java.lang.reflect.Proxy;
4
5 /**
6  * PROXY ENGENDRE A L'EXECUTION, qui imite le principe d'un stub RMI :
7  * il ne fait pas le travail, il transforme l'appel en octets et le
8  * confie a un "squelette".
9  *
10 * SIMULATION PEDAGOGIQUE : tout se passe dans la meme JVM, sans reseau.
11 * Un vrai stub RMI envoie ces octets sur une connexion TCP (JRMP).
12 */
```

```

13 public final class StubSimule {
14
15     private StubSimule() { }
16
17     @SuppressWarnings("unchecked")
18     public static <T> T pour(Class<T> contrat, SqueletteSimule squelette) {
19         return (T) Proxy.newProxyInstance(
20             contrat.getClassLoader(),
21             new Class<?>[] { contrat },
22             (proxy, methode, args) -> {
23                 Requete r = new Requete(methode.getName(),
24                                         methode.getParameterTypes(), args);
25                 byte[] message = Octets.versOctets(r); // empaquetage
26                 System.out.println("    [stub] " + r.methode() + " -> "
27                                   + message.length + " octets envoyes");
28                 byte[] reponse = squelette.recevoir(message); // "reseau"
29                 return Octets.depuisOctets(reponse); // depaquetage
30             });
31     }
32 }

```

Listing 8 – SqueletteSimule.java

```

1 package patterns.proxy;
2
3 import java.lang.reflect.Method;
4
5 /**
6  * Cote "serveur" de la simulation : recoit des octets, retrouve la
7  * methode, appelle l'objet reel, renvoie le resultat en octets.
8  * C'est le role que le runtime RMI tient pour vous.
9  */
10 public class SqueletteSimule {
11
12     private final Object objetReel;
13
14     public SqueletteSimule(Object objetReel) {
15         this.objetReel = objetReel;
16     }
17
18     public byte[] recevoir(byte[] message) throws Exception {
19         Requete r = (Requete) Octets.depuisOctets(message);
20         Method m = objetReel.getClass().getMethod(r.methode(), r.types());
21         Object resultat = m.invoke(objetReel, r.arguments());
22         return Octets.versOctets(resultat);
23     }
24 }

```

Enfin, le scénario. Le client `tableauDeBord` est **identique** dans les trois parties :

Listing 9 – DemoProxy.java

```

1 package patterns.proxy;
2
3 import java.lang.reflect.Proxy;
4
5 public class DemoProxy {
6
7     /** Le code client : il ne connaît QUE l'interface ServiceMeteo. */
8     static void tableauDeBord(ServiceMeteo service, String... villes) {

```

```

9      for (String v : villes) {
10         Prevision p = service.prevision(v);
11         System.out.println("  " + p.ville() + " : "
12             + p.celsius() + " C, " + p.ciel());
13     }
14 }
15
16 public static void main(String[] args) {
17     System.out.println("== 1. Sans proxy : chaque appel coute un calcul ==");
18     ServiceMeteoReel reel = new ServiceMeteoReel();
19     tableauDeBord(reel, "Lyon", "Lyon", "Sete");
20     System.out.println("  calculs effectues : " + reel.calculs());
21
22     System.out.println();
23     System.out.println("== 2. Proxy ecrit a la main : "
24         + "meme client, acces controle ==");
25     ServiceMeteoReel reel2 = new ServiceMeteoReel();
26     ServiceMeteo avecCache = new ProxyCacheMeteo(reel2);
27     tableauDeBord(avecCache, "Lyon", "Lyon", "Sete");
28     System.out.println("  calculs effectues : " + reel2.calculs());
29
30     System.out.println();
31     System.out.println("== 3. Proxy engendre a l'execution, "
32         + "facon stub (simulation) ==");
33     // cote "serveur" : le service reel, derriere son cache
34     ServiceMeteo coteServeur = new ProxyCacheMeteo(new ServiceMeteoReel());
35     ServiceMeteo stub =
36         StubSimule.pour(ServiceMeteo.class, new SqueletteSimule(coteServeur));
37     System.out.println("  classe du stub      : " + stub.getClass().getName());
38     System.out.println("  proxy dynamique ?   : "
39         + Proxy.isProxyClass(stub.getClass()));
40     System.out.println("  implemente le contrat ? " + (stub instanceof ServiceMeteo));
41     tableauDeBord(stub, "Lyon");
42     Prevision a = stub.prevision("Sete");
43     Prevision b = stub.prevision("Sete"); // le serveur renvoie le MEME objet
44     System.out.println("  a.equals(b) : " + a.equals(b) + "   a == b : " + (a == b)
45         + "   -> le client recoit des copies");
46 }
47 }

```

2.4. Exécution

```

== 1. Sans proxy : chaque appel coute un calcul ==
  [service reel] calcul couteux pour Lyon
  Lyon : 18 C, nuageux
  [service reel] calcul couteux pour Lyon
  Lyon : 18 C, nuageux
  [service reel] calcul couteux pour Sete
  Sete : 21 C, ensoleille
  calculs effectues : 3

== 2. Proxy ecrit a la main : meme client, acces controle ==
  [service reel] calcul couteux pour Lyon
  Lyon : 18 C, nuageux
  [proxy] Lyon servie depuis le cache
  Lyon : 18 C, nuageux
  [service reel] calcul couteux pour Sete
  Sete : 21 C, ensoleille
  calculs effectues : 2

```

```

== 3. Proxy engendre a l'execution, facon stub (simulation) ==
classe du stub      : jdk.proxy1.$Proxy0
proxy dynamique ?   : true
implémente le contrat ? true
  [stub] prevision -> 268 octets envoyes
  [service reel] calcul couteux pour Lyon
Lyon : 18 C, nuageux
  [stub] prevision -> 268 octets envoyes
  [service reel] calcul couteux pour Sete
  [stub] prevision -> 268 octets envoyes
  [proxy] Sete servie depuis le cache
a.equals(b) : true   a == b : false   -> le client recoit des copies

```

2.5. Explication pas à pas

- Partie 1.** Le client appelle directement l'objet réel : trois appels, **trois calculs**, dont un pour une ville déjà connue.
- Partie 2.** On remplace, *à la construction seulement*, `reel` par `new ProxyCacheMeteo(reel2)`. Le second « Lyon » est servi par le proxy : **deux calculs**. Le code du tableau de bord n'a pas changé d'une ligne.
- Partie 3.** `StubSimule.pour(...)` fabrique une classe `jdk.proxyN.$ProxyM` qui implémente `ServiceMeteo` : le client peut l'utiliser comme n'importe quelle implémentation.
- À chaque appel, le gestionnaire (`InvocationHandler`) reçoit la `Method` et les arguments, construit une `Requete`, la sérialise en **octets** et l'envoie au squelette.
- Le squelette désérialise, retrouve la méthode par réflexion, appelle l'objet réel et renvoie le résultat sérialisé.
- Côté serveur, le second `prevision("Sete")` est servi par le cache : c'est *le même objet*. Pourtant, côté client, `a == b` vaut `false` : chaque réponse a été désérialisée, le client détient **deux copies** égales mais distinctes.

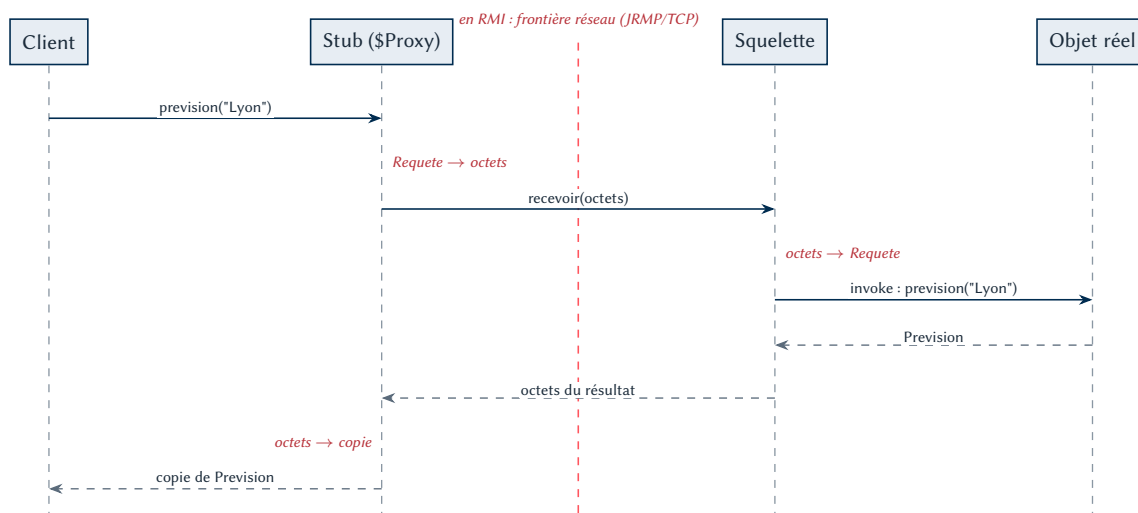


FIGURE 2 – Un appel à travers le stub simulé (partie 3)

2.6. Ce que le patron permet de faire évoluer

- Ajouter une politique d'accès** — cache, contrôle de droits, comptage, chargement paresseux — sans modifier ni le client ni l'objet réel.
- Retirer ou remplacer** cette politique en changeant une seule ligne : celle qui construit l'objet passé au client.
- Déplacer l'objet réel ailleurs** : si le représentant sait transporter l'appel, le client ne voit pas la différence de *syntaxe*. C'est exactement ce qu'exploite un intergiciel.

2.7. Du patron objet à l'intergiciel

A. En conception objet. Le proxy est une classe ordinaire, écrite à la main (`ProxyCacheMeteo`), qui partage le contrat de l'objet réel et s'interpose. Tout se passe en mémoire, les appels ne peuvent pas échouer pour des raisons de réseau.

B. Dans un intergiciel. Le représentant tient l'objet réel *dans un autre processus*. Son travail devient : emballer l'appel, le transmettre, attendre, déballer le résultat. Et il n'est plus écrit à la main : il est **engendré**, parce que sa structure est uniforme pour n'importe quel contrat (CM 01, patron 1).

RÔLE GOF	EN JAVA RMI	PRÉCISION
Subject	l'interface distante (<code>extends Remote</code>)	ses méthodes déclarent <code>RemoteException</code>
RealSubject	le servant (<code>UnicastRemoteObject</code>)	reste dans la JVM du serveur
Proxy	le stub	engendré à l'exportation ; obtenu par <code>lookup</code>
Client	le client RMI	ne connaît que l'interface

ANALOGIE ET DÉTAILS D'IMPLÉMENTATION

L'analogie avec `StubSimule` est structurelle, pas littérale :

- **Ce qui est identique.** Depuis Java 5, le stub RMI est bien un `java.lang.reflect.Proxy` : `stub.getClass().getName()` affiche `jdk.proxyN.$ProxyM` (CM 02, diapositive 24 ; point de contrôle A du TP 1). Son gestionnaire est un `RemoteObjectInvocationHandler`, et les paramètres sont emballés par la **sérialisation Java**.
- **Ce qui diffère.** Le vrai stub transporte l'appel sur une connexion TCP (protocole JRMP) jusqu'à une autre JVM ; il désigne la méthode par une empreinte de sa signature, pas par son nom ; il remplace un paramètre `Remote` par un stub au lieu de le copier ; et un échec réseau devient une `RemoteException`. Le « squelette » n'est plus une classe engendrée : c'est le runtime RMI qui assure ce rôle.

La partie 3 montre donc *pourquoi* le client RMI reçoit des copies des objets sérialisables — c'est exactement l'étape A2 du TP 1.

Dans l'UE. CM 01 §4, patron 1 · CM 02 §1 et §4 (le stub, diapositive 24) · TP 1, point de contrôle A · TP 4 : le « client » gRPC engendré depuis un fichier `.proto` joue le même rôle.

2.8. Pièges et confusions

- « **Un proxy, c'est `java.lang.reflect.Proxy`** ». Non : `ProxyCacheMeteo` est un proxy parfaitement valable, sans aucune réflexion. Le proxy dynamique n'est qu'une *réalisation* du patron, utile quand le contrat n'est pas connu à l'écriture du code.
- « **Un proxy local est un stub** ». Non : sans transport ni second processus, il n'y a rien de distribué. La partie 3 est une simulation.
- **La transparence a un prix.** Un appel sur un stub ressemble à un appel local, mais il peut être mille fois plus lent et échouer à tout moment. C'est pourquoi RMI *impose* `RemoteException` dans le contrat : le client ne peut pas l'ignorer.
- **`toString`, `equals` et `hashCode` passent aussi par le gestionnaire** d'un proxy dynamique. Dans `StubSimule`, appeler `stub.toString()` déclencherait un aller-retour complet vers le squelette. Un vrai stub RMI traite ces méthodes localement.

CODE SOURCE

`resources/design-patterns/proxy — java -cp out patterns.proxy.DemoProxy`

3. Adapter — faire parler deux contrats incompatibles

3.1. Le problème

Une application doit envoyer des rappels de rendez-vous. Elle raisonne avec son propre vocabulaire : un *destinataire* (un nom), un *message* (du texte), et une erreur métier si l'envoi échoue.

Le seul moyen d'envoi disponible est une passerelle SMS **existante**, que l'on ne peut pas modifier : bibliothèque tierce, service d'un autre système. Son contrat n'a rien à voir avec le nôtre :

CE QUE VEUT L'APPLICATION	CE QUE PROPOSE LA PASSERELLE	CONVERSION NÉCESSAIRE
un nom : "alice"	un numéro international : "+336..."	consulter un annuaire
du texte	des octets (byte[])	encoder en UTF-8
une exception métier	un code de retour (0, 17, 42)	traduire chaque code
notifier(dest, msg)	sendSms(e164, payload, flags)	renommer, compléter flags

La solution naïve (partie 1 de l'exemple) fait ces conversions *là où l'on notifie*. Elle fonctionne — une fois. Au troisième endroit du code qui envoie un rappel, les conversions sont recopiées, les codes d'erreur traités différemment, et le jour où la passerelle change, il faut tout retrouver.

3.2. Intention et participants

INTENTION

Convertir l'interface d'un composant existant en l'interface attendue par le client, sans modifier ni l'un ni l'autre. Toutes les traductions sont concentrées en un seul endroit.

RÔLE	DANS L'EXEMPLE	RESPONSABILITÉ
Target	Notificateur	l'interface attendue par le client
Adaptee	PasserelleSmsHeritee	le composant existant, au contrat incompatible
Adapter	AdaptateurSms	implémente Target; détient une référence vers Adaptee; traduit
Client	rappelerRendezVous(...)	ne dépend que de Target

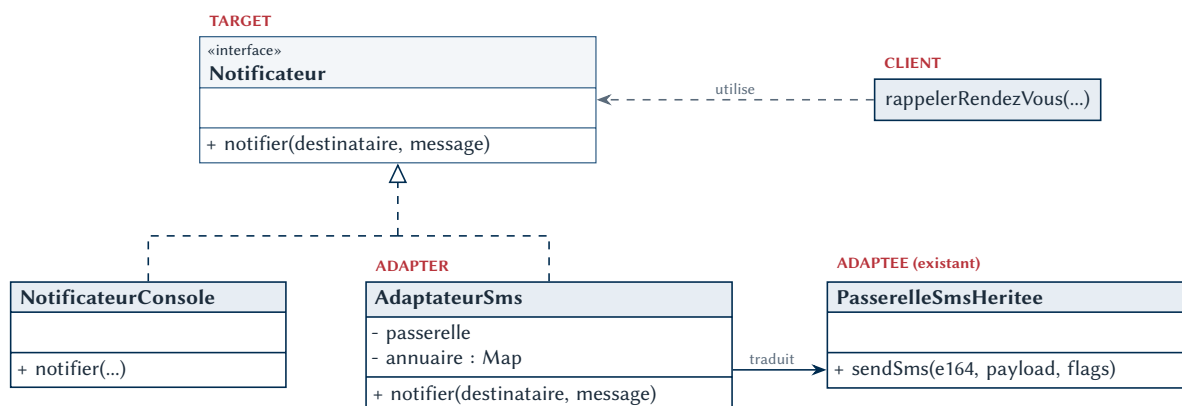


FIGURE 3 – Structure du patron Adapter (adaptateur d'objet)

3.3. L'exemple, fichier par fichier

Le contrat attendu et son exception métier :

Listing 10 – Notificateur.java

```
1 package patterns.adapter;
2
3 /**
4  * TARGET : l'interface que NOTRE application attend.
5  * Vocabulaire du domaine, une exception metier, aucun detail technique.
6  */
7 public interface Notificateur {
8
9     void notifier(String destinataire, String message) throws NotificationEchouee;
10 }
```

Listing 11 – NotificationEchouee.java

```
1 package patterns.adapter;
2
3 /** Exception du domaine : l'application ne voit jamais un code d'erreur brut. */
4 public class NotificationEchouee extends Exception {
5
6     private static final long serialVersionUID = 1L;
7
8     public NotificationEchouee(String message) { super(message); }
9 }
```

Le composant existant. Ses noms anglais et ses codes de retour rappellent qu'il n'a pas été écrit pour notre application :

Listing 12 – PasserelleSmsHeritee.java

```
1 package patterns.adapter;
2
3 import java.nio.charset.StandardCharsets;
4
5 /**
6  * ADAPTEE : un composant EXISTANT que l'on ne peut pas modifier
7  * (bibliotheque tierce, service d'un autre systeme...).
8  *
9  * Son contrat ne ressemble pas au notre : numero international au lieu
10 * d'un nom, octets au lieu de texte, code de retour au lieu d'exception.
11 */
12 public class PasserelleSmsHeritee {
13
14     public static final int OK = 0;
15     public static final int ERR_NUMBER = 17;
16     public static final int ERR_TOO_LONG = 42;
17
18     public int sendSms(String e164Number, byte[] payload, int flags) {
19         if (!e164Number.startsWith("+")) return ERR_NUMBER;
20         if (payload.length > 160) return ERR_TOO_LONG;
21         System.out.println("    [passerelle] SMS vers " + e164Number + " ("
22             + payload.length + " octets) : "
23             + new String(payload, StandardCharsets.UTF_8));
24         return OK;
25     }
26 }
```

L'adaptateur : **toutes** les conversions du tableau précédent, et seulement elles.

Listing 13 – AdaptateurSms.java

```

1  package patterns.adapter;
2
3  import java.nio.charset.StandardCharsets;
4  import java.util.Map;
5
6  /**
7   * ADAPTER : implemente l'interface attendue (Notificateur) et traduit
8   * chaque appel vers l'interface existante (PasserelleSmsHeritee).
9   *
10  * Toutes les conversions vivent ICI, et seulement ici :
11  *   nom -> numero, texte -> octets, code d'erreur -> exception metier.
12  */
13  public class AdaptateurSms implements Notificateur {
14
15      private final PasserelleSmsHeritee passerelle;
16      private final Map<String, String> annuaire;
17
18      public AdaptateurSms(PasserelleSmsHeritee passerelle, Map<String, String> annuaire) {
19          this.passerelle = passerelle;
20          this.annuaire = annuaire;
21      }
22
23      @Override
24      public void notifier(String destinataire, String message) throws NotificationEchouee {
25          String numero = annuaire.get(destinataire);
26          if (numero == null)
27              throw new NotificationEchouee("destinataire inconnu : " + destinataire);
28
29          int code = passerelle.sendSms(numero,
30              message.getBytes(StandardCharsets.UTF_8), 0);
31
32          switch (code) {
33              case PasserelleSmsHeritee.OK -> { }
34              case PasserelleSmsHeritee.ERR_TOO_LONG ->
35                  throw new NotificationEchouee("message trop long pour un SMS");
36              default ->
37                  throw new NotificationEchouee(
38                      "envoi refuse par la passerelle (code " + code + ")");
39          }
40      }
41  }

```

Une autre réalisation du même contrat, qui n'a besoin d'aucune adaptation, puis le scénario :

Listing 14 – NotificateurConsole.java

```

1  package patterns.adapter;
2
3  /** Une autre realisation du meme contrat, sans aucune adaptation. */
4  public class NotificateurConsole implements Notificateur {
5
6      @Override
7      public void notifier(String destinataire, String message) {
8          System.out.println("    [console] a " + destinataire + " : " + message);
9      }
10 }

```

Listing 15 – DemoAdapter.java

```

1 package patterns.adapter;
2
3 import java.nio.charset.StandardCharsets;
4 import java.util.Map;
5
6 public class DemoAdapter {
7
8     static final Map<String, String> ANNUAIRE = Map.of("alice", "+33600000001");
9
10    /** Le code applicatif : il ne depend QUE de Notificateur. */
11    static void rappelerRendezVous(Notificateur n, String qui) {
12        try {
13            n.notifier(qui, "Rappel : rendez-vous demain a 10h");
14            System.out.println("  rappel envoye a " + qui);
15        } catch (NotificationEchouee e) {
16            System.out.println("  rappel impossible : " + e.getMessage());
17        }
18    }
19
20    public static void main(String[] args) {
21        System.out.println("== 1. Sans adaptateur : "
22            + "les conversions envahissent l'application ==");
23        PasserelleSmsHeritee passerelle = new PasserelleSmsHeritee();
24        String numero = ANNUAIRE.get("alice");
25        int code = passerelle.sendSms(numero,
26            "Rappel : rendez-vous demain a 10h".getBytes(StandardCharsets.UTF_8), 0);
27        if (code != PasserelleSmsHeritee.OK)
28            System.out.println("  code d'erreur " + code);
29        System.out.println("  (a recopier partout ou l'on veut notifier)");
30
31        System.out.println();
32        System.out.println("== 2. Avec adaptateur : "
33            + "l'application parle son propre langage ==");
34        Notificateur sms = new AdaptateurSms(passerelle, ANNUAIRE);
35        rappelerRendezVous(sms, "alice");
36        rappelerRendezVous(sms, "bob");
37
38        System.out.println();
39        System.out.println("== 3. Le meme code applicatif, "
40            + "une autre realisation du contrat ==");
41        rappelerRendezVous(new NotificateurConsole(), "bob");
42    }
43 }

```

3.4. Exécution

```
== 1. Sans adaptateur : les conversions envahissent l'application ==
[passerelle] SMS vers +33600000001 (33 octets) : Rappel : rendez-vous demain a 10h
(a recopier partout ou l'on veut notifier)

== 2. Avec adaptateur : l'application parle son propre langage ==
[passerelle] SMS vers +33600000001 (33 octets) : Rappel : rendez-vous demain a 10h
rappel envoye a alice
rappel impossible : destinataire inconnu : bob

== 3. Le meme code applicatif, une autre realisation du contrat ==
[console] a bob : Rappel : rendez-vous demain a 10h
rappel envoye a bob
```

3.5. Explication pas à pas

- Partie 1.** Le code applicatif manipule directement la passerelle : annuaire, octets, code de retour. Tout cela serait à recopier partout.
- Partie 2.** `rappelerRendezVous` ne connaît que `Notificateur`. L'envoi à `alice` traverse l'adaptateur jusqu'à la passerelle; pour `bob`, absent de l'annuaire, l'adaptateur lève une `NotificationEchouee` au message compréhensible.
- Partie 3.** Le *même* `rappelerRendezVous` fonctionne avec `NotificateurConsole` : l'application est découplée du moyen d'envoi.

3.6. Ce que le patron permet de faire évoluer

- **Changer de fournisseur** : une nouvelle passerelle demande un nouvel adaptateur, pas une modification de l'application.
- **Faire évoluer le contrat de l'application** indépendamment de celui des composants existants.
- **Tester** l'application avec une réalisation simple du contrat, sans dépendre de la passerelle réelle.

3.7. Du patron objet à l'intergiciel

A. En conception objet. Un adaptateur d'objet est une classe qui implémente une interface et délègue à un objet d'une autre interface, en convertissant noms, types et erreurs.

B. Dans un intergiciel. Le même principe apparaît à deux échelles.

- **Adaptateurs de protocole et passerelles** (CM 01, patron 2) : exposer un système existant derrière une interface nouvelle — un service SOAP devant une application ancienne, une façade REST devant un service interne. C'est exactement la situation des TP 2 et TP 3.
- **Côté client**, les classes que vous manipulez sont souvent *techniques* : un stub dont chaque méthode lève `RemoteException`, des classes engendrées depuis un WSDL ou un fichier `.proto`. Placer une couche qui présente au reste de l'application une interface *du domaine*, et y traduire les erreurs techniques en erreurs compréhensibles, évite que ces détails se répandent partout — jusque dans l'interface utilisateur.

Dans l'UE. CM 01 §4, patron 2 · TP 1 : la séparation entre la présentation (CLI) et la logique client qui manipule les interfaces distantes relève de la même idée · TP 2 et TP 3 : passerelles entre technologies d'appel distant.

3.8. Pièges et confusions

- **Adapter ou Proxy?** Les deux s'interposent et délèguent. Le proxy *garde* l'interface de l'objet qu'il représente; l'adaptateur la *change*.
- **L'adaptateur qui grossit.** Il traduit; il ne décide pas. Dès qu'il contient des règles métier, ce n'est plus un adaptateur mais un service mal placé.

- **Traduire une erreur n'est pas la faire disparaître.** À distance, un délai dépassé ne prouve pas que l'opération n'a pas eu lieu. Une exception métier peut porter un message plus clair, mais elle ne doit pas masquer cette incertitude à l'appelant.

CODE SOURCE

```
resources/design-patterns/adapter — java -cp out patterns.adapter.DemoAdapter
```

4. Factory — confier la création à qui sait la décider

4.1. Le problème

Une messagerie envoie des messages sur le réseau. Selon la configuration, ils sont transmis **en texte** (lisible, volumineux) ou **compressés** (compacts, illisibles). Deux objets sont nécessaires : un *encodeur* chez l'émetteur, un *décodeur* chez le récepteur — et ils **doivent aller ensemble**.

La solution naïve écrit `new` partout où l'on a besoin d'un encodeur ou d'un décodeur. Deux défauts apparaissent aussitôt :

- **la décision est dispersée** : changer de format oblige à retrouver chaque `new` ;
- **rien ne garantit la cohérence** : un encodeur compressé associé à un décodeur texte compile parfaitement, et produit un message illisible. C'est ce que montre la partie 1 de l'exemple.

4.2. Intention et vocabulaire

INTENTION — « FACTORY » AU SENS DU CM 01

Séparer la décision de création de l'utilisation. Le client demande des objets à une *fabrique abstraite* ; ce sont les *fabriques concrètes* qui décident quelles classes instancier. Quand la fabrique crée plusieurs objets liés, elle garantit qu'ils appartiennent à la **même famille**.

Trois termes voisins sont souvent confondus. Le CM 01 (diapositive 32) présente la fabrique abstraite, réalisée par des méthodes de fabrique ; l'exemple les montre toutes deux, et signale le troisième terme pour ne pas le confondre :

TERME	DÉFINITION	DANS L'EXEMPLE
Abstract Factory (patron GoF)	une interface qui crée une <i>famille</i> de produits liés, sans nommer leurs classes concrètes	<code>FabriqueDeCodec</code>
Factory Method (patron GoF)	une méthode de création déclarée dans un type, dont la classe instanciée est décidée par une sous-classe ou une réalisation	<code>creerEncodeur()</code> , <code>creerDecodeur()</code>
Fabrique simple (idiome, pas un patron GoF)	une méthode, souvent statique, qui choisit quoi créer d'après un paramètre	<code>FabriqueDeCodec.selon("...")</code>

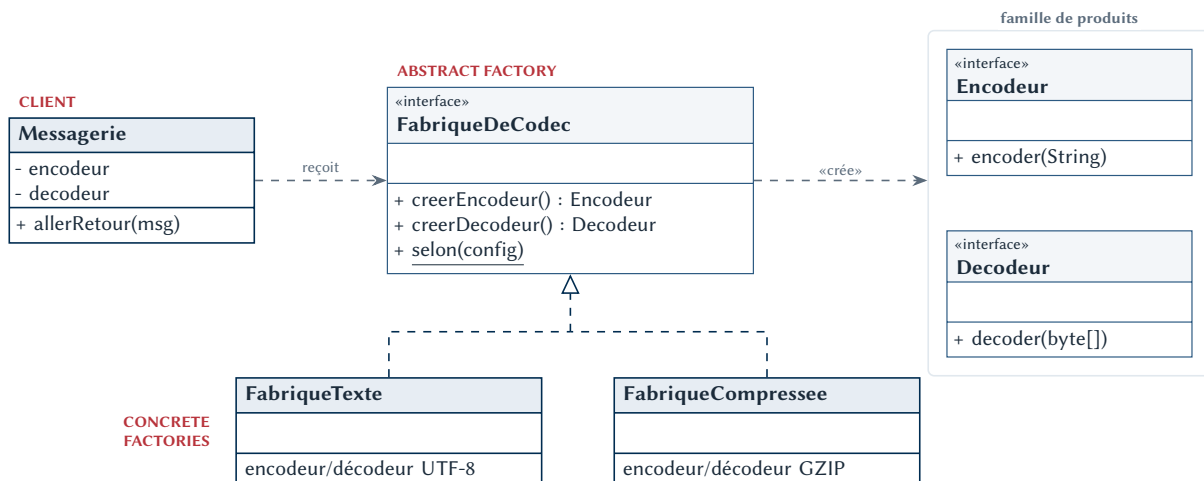


FIGURE 4 – Structure : une fabrique abstraite, deux familles de produits

4.3. L'exemple, fichier par fichier

Les deux produits, définis par leur seul contrat :

Listing 16 – Encodeur.java

```

1 package patterns.factory;
2
3 /** PRODUIT A : transforme un message en octets a envoyer. */
4 public interface Encodeur {
5     byte[] encoder(String message) throws Exception;
6 }
  
```

Listing 17 – Decodeur.java

```

1 package patterns.factory;
2
3 /** PRODUIT B : reconstruit le message a partir des octets recus. */
4 public interface Decodeur {
5     String decoder(byte[] octets) throws Exception;
6 }
  
```

La fabrique abstraite. Ses deux méthodes sont des méthodes de fabrique ; la méthode statique `selon` est le *seul* endroit du programme qui associe un nom de configuration à une famille :

Listing 18 – FabriqueDeCodec.java

```

1 package patterns.factory;
2
3 /**
4  * FABRIQUE ABSTRAITE : cree une FAMILLE de produits qui doivent aller
5  * ensemble. Un encodeur et son decodeur ne se melangent pas.
6  *
7  * Chacune de ses deux methodes est une METHODE DE FABRIQUE : la classe
8  * concrete a instancier est decidee par la sous-classe.
9  */
10 public interface FabriqueDeCodec {
11
12     Encodeur creerEncodeur();
13
14     Decodeur creerDecodeur();
15
16     String nom();
  
```

```

17
18     /**
19      * Point UNIQUE ou l'on choisit la famille, a partir d'une
20      * configuration. (Fabrique simple : un choix, pas un patron GoF.)
21      */
22     static FabriqueDeCodec selon(String configuration) {
23         return switch (configuration) {
24             case "texte"      -> new FabriqueTexte();
25             case "compresse" -> new FabriqueCompressee();
26             default -> throw new IllegalArgumentException(
27                 "codec inconnu : " + configuration);
28         };
29     }
30 }

```

Les deux fabriques concrètes. Chacune livre une paire **cohérente** :

Listing 19 – FabriqueTexte.java

```

1  package patterns.factory;
2
3  import java.nio.charset.StandardCharsets;
4
5  /** FABRIQUE CONCRETE 1 : texte UTF-8, lisible, volumineux. */
6  public class FabriqueTexte implements FabriqueDeCodec {
7
8      @Override
9      public Encodeur creerEncodeur() {
10         return message -> message.getBytes(StandardCharsets.UTF_8);
11     }
12
13     @Override
14     public Decodeur creerDecodeur() {
15         return octets -> new String(octets, StandardCharsets.UTF_8);
16     }
17
18     @Override
19     public String nom() { return "texte"; }
20 }

```

Listing 20 – FabriqueCompressee.java

```

1  package patterns.factory;
2
3  import java.io.ByteArrayInputStream;
4  import java.io.ByteArrayOutputStream;
5  import java.nio.charset.StandardCharsets;
6  import java.util.zip.GZIPInputStream;
7  import java.util.zip.GZIPOutputStream;
8
9  /** FABRIQUE CONCRETE 2 : GZIP, compact, illisible sans le bon decodeur. */
10 public class FabriqueCompressee implements FabriqueDeCodec {
11
12     @Override
13     public Encodeur creerEncodeur() {
14         return message -> {
15             ByteArrayOutputStream buffer = new ByteArrayOutputStream();
16             try (GZIPOutputStream gz = new GZIPOutputStream(buffer)) {
17                 gz.write(message.getBytes(StandardCharsets.UTF_8));
18             }
19         }
20     }
21 }

```

```

19         return buffer.toByteArray();
20     };
21 }
22
23 @Override
24 public Decodeur creerDecodeur() {
25     return octets -> {
26         try (GZIPInputStream gz =
27             new GZIPInputStream(new ByteArrayInputStream(octets))) {
28             return new String(gz.readAllBytes(), StandardCharsets.UTF_8);
29         }
30     };
31 }
32
33 @Override
34 public String nom() { return "compresse"; }
35 }

```

Le client, qui n'écrit jamais `new` sur une classe concrète, puis le scénario :

Listing 21 – `Messagerie.java`

```

1  package patterns.factory;
2
3  /**
4   * CLIENT : il envoie et recoit des messages sans jamais ecrire "new" sur
5   * une classe concrete. Il ne connait que la fabrique abstraite et les
6   * interfaces des produits.
7   */
8  public class Messagerie {
9
10     private final Encodeur encodeur;
11     private final Decodeur decodeur;
12
13     public Messagerie(FabriqueDeCodec fabrique) {
14         this.encodeur = fabrique.creerEncodeur();
15         this.decodeur = fabrique.creerDecodeur();
16     }
17
18     /** Aller-retour : ce qui est envoye doit etre relu a l'identique. */
19     public String allerRetour(String message) throws Exception {
20         byte[] surLeReseau = encodeur.encoder(message);
21         System.out.println("    " + surLeReseau.length + " octets transmis");
22         return decodeur.decoder(surLeReseau);
23     }
24 }

```

Listing 22 – `DemoFactory.java`

```

1  package patterns.factory;
2
3  public class DemoFactory {
4
5     static final String MESSAGE = "bonjour bonjour bonjour bonjour bonjour bonjour";
6
7     public static void main(String[] args) throws Exception {
8         System.out.println("== 1. Sans fabrique : "
9             + "deux decisions de creation, prises separement ==");
10         Encodeur enc = new FabriqueCompressee().creerEncodeur(); // choisi ici...
11         Decodeur dec = new FabriqueTexte().creerDecodeur();       // ...et oublie la

```

```

12     byte[] octets = enc.encoder(MESSAGE);
13     String recu = dec.decoder(octets);
14     System.out.println(" message relu identique ? " + MESSAGE.equals(recu));
15
16     System.out.println();
17     System.out.println("== 2. Avec fabrique abstraite : "
18         + "une seule decision, une famille coherente ==");
19     for (String config : new String[] { "texte", "comprese" }) {
20         FabriqueDeCodec fabrique = FabriqueDeCodec.selon(config);
21         System.out.println(" famille " + fabrique.nom() + " :");
22         String relu = new Messagerie(fabrique).allerRetour(MESSAGE);
23         System.out.println(" message relu identique ? " + MESSAGE.equals(relu));
24     }
25
26     System.out.println();
27     System.out.println("== 3. Configuration inconnue : "
28         + "l'erreur est detectee a la creation ==");
29     try {
30         FabriqueDeCodec.selon("chiffre");
31     } catch (IllegalArgumentException e) {
32         System.out.println(" " + e.getMessage());
33     }
34 }
35 }

```

4.4. Exécution

```

== 1. Sans fabrique : deux decisions de creation, prises separement ==
message relu identique ? false

== 2. Avec fabrique abstraite : une seule decision, une famille coherente ==
famille texte :
  47 octets transmis
  message relu identique ? true
famille compresse :
  31 octets transmis
  message relu identique ? true

== 3. Configuration inconnue : l'erreur est detectee a la creation ==
codec inconnu : chiffre

```

4.5. Explication pas à pas

- Partie 1.** L'encodeur et le décodeur sont choisis à deux endroits différents. Le code compile, s'exécute sans erreur... et le message relu n'est pas celui qui a été envoyé.
- Partie 2.** `Messagerie` reçoit *une* fabrique et lui demande *ses deux* produits. L'incohérence de la partie 1 est devenue impossible à écrire. Changer de format revient à changer la chaîne passée à `selon`.
- Le résultat est vérifiable : 47 octets en texte, 31 octets compressés, et un message relu identique dans les deux cas.
- Partie 3.** Une configuration inconnue est refusée *au moment de la création*, en un seul endroit, avec un message clair — et non plus tard, au premier message illisible.

4.6. Ce que le patron permet de faire évoluer

- Ajouter une famille** (par exemple chiffrée) : une nouvelle fabrique concrète et une ligne dans `selon`. Ni `Messagerie` ni les familles existantes ne changent.

- **Substituer une famille de test** — un transport en mémoire, un encodage lisible pour le débogage — sans toucher au code qui l'utilise.
- **Paramétrer la création** (taille de tampon, niveau de compression) dans la fabrique, loin du code métier.

4.7. Du patron objet à l'intergiciel

A. En conception objet. La fabrique déplace la décision « quelle classe instancier ? » vers un objet dédié, remplaçable, et garantit la cohérence des objets créés ensemble.

B. Dans un intergiciel. Deux problèmes rendent la fabrique indispensable (CM 01, patron 3) :

- **On ne peut pas créer un objet « là-bas » avec `new`.** Un objet exécuté dans un autre processus doit être créé *par quelqu'un qui s'y trouve* : un objet distant dont une méthode crée d'autres objets et en renvoie les stubs joue le rôle de fabrique : le client obtient un objet distant sans jamais l'instancier lui-même.
- **L'infrastructure de communication doit rester cohérente.** Java RMI permet d'exporter un objet avec une *paire* de fabriques de sockets, `RMIClientSocketFactory` et `RMI ServerSocketFactory` (paquetage `java.rmi.server`), par exemple `SslRMIClientSocketFactory` et `SslRMIServerSocketFactory` pour du RMI sur TLS. Mélanger une fabrique client TLS et une fabrique serveur en clair produit exactement le défaut de la partie 1 : une connexion qui n'aboutit pas.

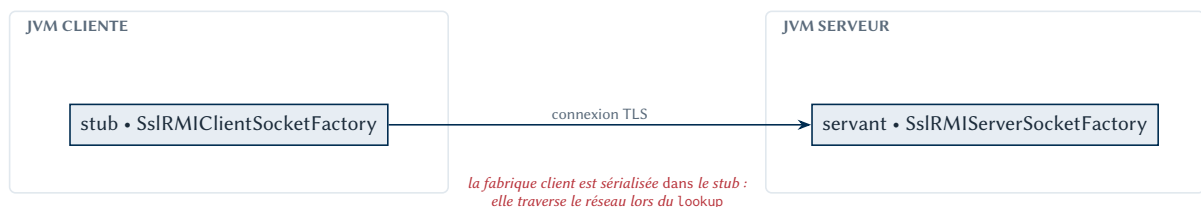


FIGURE 5 – Une famille de fabriques de sockets doit être cohérente des deux côtés

Dans l'UE. CM 01 §4, patron 3 · CM 02 §7 (fabriques de sockets).

4.8. Pièges et confusions

- **Appeler « Factory » n'importe quelle méthode statique qui fait `new`.** Une fabrique simple est utile, mais le patron Abstract Factory porte sur les *familles* cohérentes, et Factory Method sur la décision déléguée à une sous-classe.
- **Déplacer le `switch` sans le supprimer.** Si chaque méthode de la fabrique teste la configuration, on a recentralisé le problème au lieu de le résoudre. Ici, la configuration n'est lue qu'une fois, dans *selon*.
- **Une fabrique de sockets ne change pas le protocole de transport.** Elle peut décorer le flux (chiffrement, compression) mais doit rendre une `java.net.Socket` : on ne remplace pas TCP par UDP.

CODE SOURCE

```
resources/design-patterns/factory — java -cp out patterns.factory.DemoFactory
```

5. Interceptor — ajouter des traitements autour de l'appel

5.1. Le problème

Un service bancaire propose deux opérations : consulter un solde, effectuer un virement. Le métier tient en quelques lignes. Mais en production, chaque appel doit aussi être **journalisé, autorisé, validé et mesuré**. Ces préoccupations sont dites *transverses* : elles concernent toutes les opérations de tous les services, et aucune n'appartient au métier. Les écrire à la main dans chaque méthode donne ceci :

Listing 23 – ServiceBancaireSansIntercepteur.java

```

1 package patterns.interceptor;
2
3 import java.util.Set;
4
5 /**
6  * CONTRE-EXEMPLE : les preoccupations transverses ecrites a la main dans
7  * chaque methode metier. Compile et fonctionne, mais :
8  * - le metier (2 lignes) est noye dans 10 lignes d'infrastructure ;
9  * - le meme code sera recopie dans chaque methode et chaque service ;
10 * - changer la politique (ex. ajouter une mesure) oblige a tout modifier.
11 */
12 public class ServiceBancaireSansIntercepteur {
13
14     private final ServiceBancaireImpl metier = new ServiceBancaireImpl();
15
16     public void virer(String utilisateur, String depuis, String vers, long montant) {
17         System.out.println("  -> virer"); // journal
18         if (!Set.of("alice").contains(utilisateur)) // securite
19             throw new SecurityException("acces refuse a " + utilisateur);
20         if (montant <= 0) // validation
21             throw new IllegalArgumentException("montant invalide : " + montant);
22         long debut = System.nanoTime(); // mesure
23         metier.virer(depuis, vers, montant); // METIER
24         long ms = (System.nanoTime() - debut) / 1_000_000;
25         System.out.println("    [chrono] " + ms + " ms");
26         System.out.println("  <- virer"); // journal
27     }
28 }

```

Deux lignes de métier noyées dans dix lignes d’infrastructure, qu’il faudra recopier dans *solde*, dans chaque méthode de chaque service, puis modifier partout le jour où la politique de sécurité change.

5.2. Intention et participants

INTENTION

Placer sur le **chemin de l’appel** des objets d’interposition qui exécutent un traitement *avant* et *après* l’opération — ou qui **l’interrompent** —, sans modifier ni le client ni la cible. Les intercepteurs se **composent en chaîne** et s’ajoutent ou se retirent sans toucher au métier.

PARTICIPANT	DANS L'EXEMPLE	RESPONSABILITÉ
Cible	ServiceBancaireImpl	le métier, et lui seul
Invocation	Invocation	décrit l’appel en cours; <i>proceder()</i> passe au maillon suivant
Intercepteur	Intercepteur et ses réalisations	un traitement transverse : prologue, <i>proceder()</i> , épilogue
Objet d’interposition	PipelineInterception	capte l’appel du client et le fait entrer dans la chaîne



FIGURE 6 – Une invocation traverse la chaîne à l’aller, puis au retour

5.3. L'exemple, fichier par fichier

Le contrat et la cible, débarrassée de toute infrastructure :

Listing 24 – ServiceBancaire.java

```
1 package patterns.interceptor;
2
3 /** Le contrat du service metier. */
4 public interface ServiceBancaire {
5
6     long solde(String compte);
7
8     void virer(String depuis, String vers, long montant);
9 }
```

Listing 25 – ServiceBancaireImpl.java

```
1 package patterns.interceptor;
2
3 import java.util.HashMap;
4 import java.util.Map;
5
6 /**
7  * La CIBLE : du metier, et rien que du metier.
8  * Ni journalisation, ni controle d'accès, ni mesure de temps.
9  */
10 public class ServiceBancaireImpl implements ServiceBancaire {
11
12     private final Map<String, Long> comptes = new HashMap<>(Map.of("A", 500L, "B", 100L));
13
14     @Override
15     public long solde(String compte) {
16         return comptes.getOrDefault(compte, 0L);
17     }
18
19     @Override
20     public void virer(String depuis, String vers, long montant) {
21         comptes.merge(depuis, -montant, Long::sum);
22         comptes.merge(vers, montant, Long::sum);
23         System.out.println("      [cible] virement de " + montant + " effectue");
24     }
25 }
```

Le cœur du mécanisme. Une `Invocation` connaît la chaîne et sa position; `proceder()` appelle le maillon suivant, et la cible en bout de chaîne :

Listing 26 – Intercepteur.java

```
1 package patterns.interceptor;
2
3 /**
4  * Un traitement interpose sur le chemin de l'appel.
5  * Avant proceder() : le prologue. Apres : l'epilogue.
6  */
7 @FunctionalInterface
8 public interface Intercepteur {
9
10     Object interceptor(Invocation invocation) throws Throwable;
11 }
```

Listing 27 – Invocation.java

```
1 package patterns.interceptor;
2
3 import java.lang.reflect.InvocationTargetException;
4 import java.lang.reflect.Method;
5 import java.util.List;
6 import java.util.Map;
7
8 /**
9  * UNE INVOCATION EN COURS : la methode appelee, ses arguments, des
10  * metadonnees d'appel (qui appelle ?), et la position dans la chaine.
11  *
12  * proceder() passe la main a l'intercepteur suivant ou, en bout de
13  * chaine, a la cible. Un intercepteur qui n'appelle pas proceder()
14  * INTERROMPT l'appel.
15  */
16 public class Invocation {
17
18     private final Object cible;
19     private final Method methode;
20     private final Object[] arguments;
21     private final Map<String, String> metadonnees;
22     private final List<Intercepteur> chaine;
23     private int position;
24
25     Invocation(Object cible, Method methode, Object[] arguments,
26               Map<String, String> metadonnees, List<Intercepteur> chaine) {
27         this.cible = cible;
28         this.methode = methode;
29         this.arguments = arguments == null ? new Object[0] : arguments;
30         this.metadonnees = metadonnees;
31         this.chaine = chaine;
32     }
33
34     public Object proceder() throws Throwable {
35         if (position < chaine.size()) {
36             return chaine.get(position++).interceptor(this);
37         }
38         try {
39             return methode.invoke(cible, arguments); // bout de chaine
40         } catch (InvocationTargetException e) {
41             throw e.getCause();
42         }
43     }
44 }
```

```

45     public String methode()                { return methode.getName(); }
46     public Object[] arguments()            { return arguments; }
47     public String metadonnee(String cle)   { return metadonnees.get(cle); }
48 }

```

Quatre intercepteurs réutilisables. Aucun ne connaît `ServiceBancaire` : ils ne manipulent qu'une `Invocation`.

Listing 28 – Intercepteurs.java

```

1  package patterns.interceptor;
2
3  import java.util.Arrays;
4  import java.util.Set;
5
6  /** Quatre intercepteurs reutilisables, independants du service intercepte. */
7  public final class Intercepteurs {
8
9      private Intercepteurs() { }
10
11     /** Journalisation : trace l'entree et la sortie. */
12     public static Intercepteur journalisation() {
13         return inv -> {
14             System.out.println(" -> " + inv.methode()
15                 + Arrays.toString(inv.arguments()));
16             try {
17                 Object resultat = inv.proceder();
18                 System.out.println(" <- " + inv.methode()
19                     + (resultat == null ? " termine" : " = " + resultat));
20                 return resultat;
21             } catch (Throwable t) {
22                 System.out.println(" <- " + inv.methode()
23                     + " ECHEC : " + t.getMessage());
24                 throw t;
25             }
26         };
27     }
28
29     /** Controle d'accès : n'appelle PAS proceder() si l'appelant n'est pas autorise. */
30     public static Intercepteur authentication(Set<String> autorises) {
31         return inv -> {
32             String qui = inv.metadonnee("utilisateur");
33             if (!autorises.contains(qui)) {
34                 throw new SecurityException("accès refuse a " + qui);
35             }
36             System.out.println(" [auth] " + qui + " autorise");
37             return inv.proceder();
38         };
39     }
40
41     /** Validation : refuse un montant negatif avant d'atteindre le metier. */
42     public static Intercepteur validation() {
43         return inv -> {
44             if (inv.methode().equals("virer") && (long) inv.arguments()[2] <= 0) {
45                 throw new IllegalArgumentException(
46                     "montant invalide : " + inv.arguments()[2]);
47             }
48             return inv.proceder();
49         };
50     }
51 }

```

```

52     /** Mesure : duree de tout ce qui suit dans la chaine. */
53     public static Intercepteur chronometrage() {
54         return inv -> {
55             long debut = System.nanoTime();
56             try {
57                 return inv.proceder();
58             } finally {
59                 long ms = (System.nanoTime() - debut) / 1_000_000;
60                 System.out.println("    [chrono] " + inv.methode()
61                     + " : " + ms + " ms");
62             }
63         };
64     }
65 }

```

L'objet d'interposition, puis le scénario :

Listing 29 – PipelineInterception.java

```

1  package patterns.interceptor;
2
3  import java.lang.reflect.Proxy;
4  import java.util.List;
5  import java.util.Map;
6
7  /**
8   * Construit l'objet d'INTERPOSITION : un proxy dynamique qui transforme
9   * chaque appel en Invocation et le fait traverser la chaine.
10  *
11  * Ni le client ni la cible ne sont modifies.
12  */
13  public final class PipelineInterception {
14
15      private PipelineInterception() { }
16
17      @SuppressWarnings("unchecked")
18      public static <T> T envelopper(Class<T> contrat, T cible,
19          Map<String, String> metadonnees,
20          List<Intercepteur> chaine) {
21          return (T) Proxy.newProxyInstance(
22              contrat.getClassLoader(),
23              new Class<?>[] { contrat },
24              (proxy, methode, args) -> new Invocation(
25                  cible, methode, args, metadonnees, chaine).proceder());
26      }
27  }

```

Listing 30 – DemoInterceptor.java

```

1  package patterns.interceptor;
2
3  import static patterns.interceptor.Intercepteurs.authentication;
4  import static patterns.interceptor.Intercepteurs.chronometrage;
5  import static patterns.interceptor.Intercepteurs.journalisation;
6  import static patterns.interceptor.Intercepteurs.validation;
7
8  import java.util.List;
9  import java.util.Map;
10 import java.util.Set;
11

```

```

12 public class DemoInterceptor {
13
14     static void essayer(String titre, Runnable appel) {
15         System.out.println(titre);
16         try {
17             appel.run();
18         } catch (RuntimeException e) {
19             System.out.println(" => appel rejete : " + e.getMessage());
20         }
21         System.out.println();
22     }
23
24     public static void main(String[] args) {
25         List<Intercepteur> chaine = List.of(
26             journalisation(), authentication(Set.of("alice")),
27             validation(), chronometrage());
28
29         ServiceBancaire cible = new ServiceBancaireImpl();
30         ServiceBancaire pourAlice = PipelineInterception.envelopper(
31             ServiceBancaire.class, cible, Map.of("utilisateur", "alice"), chaine);
32         ServiceBancaire pourMallory = PipelineInterception.envelopper(
33             ServiceBancaire.class, cible, Map.of("utilisateur", "mallory"), chaine);
34
35         essayer("== 1. Appel autorise : l'invocation traverse toute la chaine ==",
36             () -> pourAlice.virer("A", "B", 50));
37
38         essayer("== 2. Appelant non autorise : "
39             + "la chaine s'interrompt, la cible n'est pas atteinte ==",
40             () -> pourMallory.virer("A", "B", 50));
41
42         essayer("== 3. Argument invalide : rejete par la validation ==",
43             () -> pourAlice.virer("A", "B", -10));
44
45         List<Intercepteur> lecture = List.of(journalisation());
46         ServiceBancaire consultation = PipelineInterception.envelopper(
47             ServiceBancaire.class, cible, Map.of("utilisateur", "invite"), lecture);
48         essayer("== 4. Autre chaine, meme cible : consultation journalisee seulement ==",
49             () -> consultation.solde("B"));
50     }
51 }

```

5.4. Exécution

```

== 1. Appel autorise : l'invocation traverse toute la chaine ==
-> virer[A, B, 50]
  [auth] alice autorise
    [cible] virement de 50 effectue
    [chrono] virer : 3 ms
<- virer termine

== 2. Appelant non autorise : la chaine s'interrompt, la cible n'est pas atteinte ==
-> virer[A, B, 50]
<- virer ECHEC : acces refuse a mallory
=> appel rejete : acces refuse a mallory

== 3. Argument invalide : rejete par la validation ==
-> virer[A, B, -10]
  [auth] alice autorise
<- virer ECHEC : montant invalide : -10
=> appel rejete : montant invalide : -10

```

```

== 4. Autre chaîne, même cible : consultation journalisée seulement ==
-> solde[B]
<- solde = 150

```

La durée affichée par [chrono] varie d’une exécution à l’autre.

5.5. Explication pas à pas

- Partie 1.** L’appel entre dans le proxy, qui crée une *Invocation*. La journalisation affiche `->` puis appelle `proceeder()` ; l’authentification vérifie la métadonnée `utilisateur` et appelle `proceeder()` ; puis la validation et la mesure ; enfin la cible. Au retour, chaque intercepteur termine son épilogue : la mesure s’affiche *avant* la sortie du journal.
- Partie 2.** Pour *mallory*, l’authentification lève une exception *au lieu* d’appeler `proceeder()` : la validation, la mesure et la cible ne sont jamais atteintes. Le journal, placé avant, voit passer l’échec.
- Partie 3.** Même mécanisme pour un montant négatif, arrêté par la validation.
- Partie 4.** Une autre chaîne — journalisation seule — enveloppe *la même cible*. Le métier n’a pas changé ; seule la politique d’interposition diffère.



FIGURE 7 – Court-circuit : un intercepteur n’appelle pas `proceeder()` (partie 2)

5.6. Ce que le patron permet de faire évoluer

- **Ajouter un traitement transverse** (une métrique, un cache, une limitation de débit) en écrivant un intercepteur et en l’ajoutant à la liste : aucun service n’est modifié.
- **Choisir la politique par contexte** : chaîne complète pour les opérations sensibles, journalisation seule pour la consultation.
- **Réordonner** : placer la journalisation avant ou après l’authentification change ce qui est tracé — et c’est une décision explicite, lisible en une ligne.

5.7. Du patron objet à l’intergiciel

A. En conception objet. L’intercepteur est un objet qui encadre un appel de méthode. La réalisation par proxy dynamique n’est qu’un moyen : on peut aussi écrire des enveloppes à la main.

B. Dans un intergiciel. L’appel distant a une vie propre — une requête arrive, est décodée, traverse des traitements, atteint le servant, repart — et les intergiciels exposent précisément ce chemin sous forme de **chaîne d’intercepteurs**. Les « métadonnées » de l’exemple y deviennent les en-têtes du message : identité de l’appelant, jeton, identifiant de trace.

TECHNOLOGIE	MÉCANISME D’INTERCEPTION	DANS L’UE
SOAP (JAX-WS)	chaînes de <i>handlers</i> sur les messages	TP 2
REST (Spring)	filtres et intercepteurs de requêtes	TP 3
gRPC	<code>ClientInterceptor</code> , <code>ServerInterceptor</code>	TP 4
Spring	proxys AOP (transactions, sécurité) autour des beans	suite de l’UE
Java RMI	pas d’API d’interception publique	—

Dans l’UE. CM 01 §4, patron 4 · les TP 2 à TP 4.

5.8. Pièges et confusions

- « Appeler un logger au début de la méthode, c'est un intercepteur ». Non : le code transverse est alors *dans* la cible. L'intercepteur est *autour*, remplaçable, et la cible l'ignore.
- Oublier `proceed()`. Un intercepteur qui ne le rappelle pas interrompt silencieusement l'appel. C'est parfois voulu (sécurité); c'est souvent un bogue.
- **Intercepteur ou Proxy?** Même mécanique d'interposition, intention différente : le proxy *représente* un objet et contrôle l'accès; l'intercepteur *ajoute des traitements* le long d'un chemin, en chaîne.
- **Déballer les exceptions.** Par réflexion, une exception de la cible arrive enveloppée dans une `InvocationTargetException` : `Invocation` renvoie sa cause, pour que le client voie l'exception d'origine.

CODE SOURCE

```
resources/design-patterns/interceptor — java -cp out patterns.interceptor.DemoInterceptor
```

6. Observer — prévenir sans dépendre

6.1. Le problème

Un capteur mesure la température d'une salle de serveurs. Plusieurs modules s'intéressent à ses mesures : un tableau de bord les affiche, une alarme réagit à la surchauffe, un historique les conserve. D'autres viendront. La solution naïve fait appeler par le capteur chaque consommateur :

Listing 31 – CapteurSansObserver.java

```
1 package patterns.observer;
2
3 /**
4  * CONTRE-EXEMPLE : le capteur appelle directement chaque consommateur.
5  * - il depend de toutes les classes qui s'interessent a lui ;
6  * - ajouter un consommateur oblige a modifier le capteur ;
7  * - impossible de s'abonner ou de se desabonner a l'execution.
8  */
9 public class CapteurSansObserver {
10
11     private final Abonnes.TableauDeBord tableau = new Abonnes.TableauDeBord();
12     private final Abonnes.AlarmeSurchauffe alarme = new Abonnes.AlarmeSurchauffe(30);
13     private int numero;
14
15     public void publier(double celsius) {
16         Mesure m = new Mesure("S1", ++numero, celsius);
17         tableau.mesureRecue(m); // couplage direct
18         alarme.mesureRecue(m); // couplage direct
19         // ... et demain : historique.ajouter(m); supervision.envoyer(m); ...
20     }
21 }
```

Le capteur **dépend de tous ceux qui l'écoutent**. Ajouter un consommateur oblige à le modifier; aucun module ne peut commencer ou cesser d'écouter pendant l'exécution; et le capteur ne peut pas être réutilisé ailleurs sans emporter ses consommateurs avec lui.

6.2. Intention et participants

INTENTION

Définir une relation **un-à-plusieurs** dans laquelle le changement d'état du *sujet* est **notifié** à tous ses *observateurs*, sans que le sujet connaisse autre chose d'eux qu'une interface. Les observateurs **s'abonnent** et **se désabonnent** dynamiquement.

RÔLE	DANS L'EXEMPLE	RESPONSABILITÉ
Subject	CapteurTemperature	tient la liste des abonnés; <i>abonner</i> , <i>desabonner</i> , <i>notifie</i>
Observer	ObservateurTemperature	le seul type que le sujet connaît
ConcreteObserver	TableauDeBord, AlarmeSurchauffe, Historique	chacun décide de sa réaction
Événement	Mesure	ce qui est transmis à chaque notification

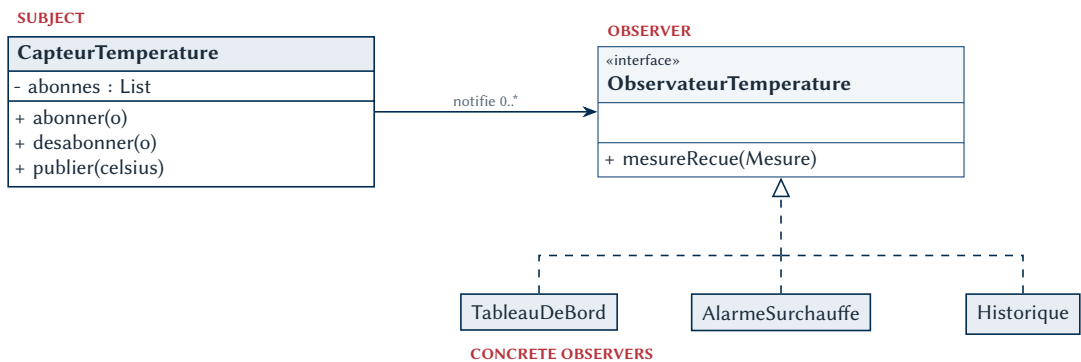


FIGURE 8 – Structure du patron Observer dans l'exemple

6.3. L'exemple, fichier par fichier

L'événement, le contrat des observateurs et le sujet :

Listing 32 – Mesure.java

```
1 package patterns.observer;
2
3 /** L'evenement publie : une mesure de temperature. */
4 public record Mesure(String salle, int numero, double celsius) { }
```

Listing 33 – ObservateurTemperature.java

```
1 package patterns.observer;
2
3 /** OBSERVER : ce que le sujet sait de ses abonnes. Rien d'autre. */
4 public interface ObservateurTemperature {
5
6     void mesureRecue(Mesure mesure);
7 }
```

Listing 34 – CapteurTemperature.java

```

1  package patterns.observer;
2
3  import java.util.List;
4  import java.util.concurrent.CopyOnWriteArrayList;
5
6  /**
7   * SUBJECT : publie ses mesures a une liste d'abonnes qu'il ne connait
8   * que par leur interface.
9   */
10 public class CapteurTemperature {
11
12     private final String salle;
13     // CopyOnWrite : un abonne peut se desabonner PENDANT une notification
14     private final List<ObservateurTemperature> abonnes = new CopyOnWriteArrayList<>();
15     private int numero;
16
17     public CapteurTemperature(String salle) {
18         this.salle = salle;
19     }
20
21     public void abonner(ObservateurTemperature o) { abonnes.add(o); }
22
23     public void desabonner(ObservateurTemperature o) { abonnes.remove(o); }
24
25     public void publier(double celsius) {
26         Mesure m = new Mesure(salle, ++numero, celsius);
27         System.out.println("[capteur] mesure " + m.numero() + " : " + celsius + " C -> "
28             + abonnes.size() + " abonne(s)");
29         for (ObservateurTemperature o : abonnes) {
30             try {
31                 o.mesureRecue(m);
32             } catch (RuntimeException e) {
33                 // un abonne defaillant ne doit pas priver les autres de la notification
34                 abonnes.remove(o);
35                 System.out.println(" [capteur] abonne defaillant retire : "
36                     + e.getMessage());
37             }
38         }
39     }
40 }

```

Trois observateurs aux réactions différentes, et un observateur défaillant :

Listing 35 – Abonnes.java

```

1  package patterns.observer;
2
3  import java.util.ArrayList;
4  import java.util.List;
5
6  /** Trois observateurs aux reactions differentes, et un observateur defaillant. */
7  public final class Abonnes {
8
9     private Abonnes() { }
10
11     /** Affiche chaque mesure. */
12     public static class TableauDeBord implements ObservateurTemperature {
13         @Override public void mesureRecue(Mesure m) {
14             System.out.println(" [tableau] " + m.salle()

```

```

15         + " : " + m.celsius() + " C");
16     }
17 }
18
19 /** Ne reagit qu'au PASSAGE au-dessus du seuil, pas a chaque mesure. */
20 public static class AlarmeSurchauffe implements ObservateurTemperature {
21     private final double seuil;
22     private boolean enAlarme;
23     public AlarmeSurchauffe(double seuil) { this.seuil = seuil; }
24     @Override public void mesureRecue(Mesure m) {
25         boolean chaud = m.celsius() >= seuil;
26         if (chaud && !enAlarme)
27             System.out.println(" [alarme]      SURCHAUFFE en " + m.salle());
28         if (!chaud && enAlarme)
29             System.out.println(" [alarme]      retour a la normale");
30         enAlarme = chaud;
31     }
32 }
33
34 /** Garde un historique, puis se desabonne seul apres n mesures. */
35 public static class Historique implements ObservateurTemperature {
36     private final List<Double> valeurs = new ArrayList<>();
37     private final CapteurTemperature capteur;
38     private final int limite;
39     public Historique(CapteurTemperature capteur, int limite) {
40         this.capteur = capteur; this.limite = limite;
41     }
42     @Override public void mesureRecue(Mesure m) {
43         valeurs.add(m.celsius());
44         if (valeurs.size() == limite) {
45             System.out.println(" [historique] " + valeurs + " -> je me desabonne");
46             capteur.desabonner(this);
47         }
48     }
49 }
50
51 /** Simule un abonne en panne. */
52 public static class ObservateurDefaillant implements ObservateurTemperature {
53     @Override public void mesureRecue(Mesure m) {
54         throw new IllegalStateException("ecran deconnecte");
55     }
56 }
57 }

```

Le scénario :

Listing 36 – DemoObserver.java

```

1  package patterns.observer;
2
3  public class DemoObserver {
4
5      public static void main(String[] args) {
6          System.out.println("== 1. Sans Observer : "
7              + "le capteur connait chaque consommateur ==");
8          CapteurSansObserver couple = new CapteurSansObserver();
9          couple.publier(24.0);
10
11          System.out.println();
12          System.out.println("== 2. Avec Observer : abonnements dynamiques ==");

```

```

13     CapteurTemperature capteur = new CapteurTemperature("S1");
14     capteur.abonner(new Abonnes.TableauDeBord());
15     capteur.abonner(new Abonnes.AlarmeSurchauffe(30));
16     capteur.abonner(new Abonnes.Historique(capteur, 2));
17     capteur.publier(24.0);
18     capteur.publier(31.5);      // l'alarme se declenche ; l'historique se desabonne
19     capteur.publier(26.0);      // plus que deux abonnes
20
21     System.out.println();
22     System.out.println("== 3. Un abonne defaillant "
23         + "n'empêche pas les autres d'être notifiés ==");
24     capteur.abonner(new Abonnes.ObservateurDefaillant());
25     capteur.publier(27.0);
26     capteur.publier(28.0);      // le defaillant a été retiré
27 }
28 }

```

6.4. Exécution

```

== 1. Sans Observer : le capteur connaît chaque consommateur ==
[tableau]    S1 : 24.0 C

== 2. Avec Observer : abonnements dynamiques ==
[capteur] mesure 1 : 24.0 C -> 3 abonne(s)
[tableau]    S1 : 24.0 C
[capteur] mesure 2 : 31.5 C -> 3 abonne(s)
[tableau]    S1 : 31.5 C
[alarme]     SURCHAUFFE en S1
[historique] [24.0, 31.5] -> je me desabonne
[capteur] mesure 3 : 26.0 C -> 2 abonne(s)
[tableau]    S1 : 26.0 C
[alarme]     retour à la normale

== 3. Un abonne defaillant n'empêche pas les autres d'être notifiés ==
[capteur] mesure 4 : 27.0 C -> 3 abonne(s)
[tableau]    S1 : 27.0 C
[capteur] abonne defaillant retire : écran deconnecté
[capteur] mesure 5 : 28.0 C -> 2 abonne(s)
[tableau]    S1 : 28.0 C

```

6.5. Explication pas à pas

- Partie 1.** Le capteur couplé appelle ses consommateurs en dur.
- Partie 2.** Trois observateurs s'abonnent. À chaque **publier**, tous sont notifiés et **chacun décide** : le tableau affiche, l'alarme ne réagit qu'au *franchissement* de 30 °C (et au retour sous le seuil), l'historique enregistre.
- Au bout de deux mesures, l'historique **se désabonne lui-même pendant la notification**. C'est possible parce que la liste est une `CopyOnWriteArrayList` : la boucle en cours n'est pas perturbée. La mesure suivante n'a plus que deux abonnés.
- Partie 3.** Un observateur lève une exception. Le sujet l'isole, **notifie quand même les autres**, puis le retire : à la mesure 5, il ne reste que deux abonnés.

6.6. Ce que le patron permet de faire évoluer

- **Ajouter un consommateur** sans modifier le sujet : écrire un observateur, l'abonner.
- **Changer les abonnements à l'exécution**, y compris depuis un observateur.
- **Réutiliser le sujet** dans un autre contexte, puisqu'il ne dépend que d'une interface.

6.7. Du patron objet à l'intergiciel

A. En conception objet. Le sujet détient des *références Java* vers ses observateurs. Une notification est un appel de méthode local : il est rapide, il ne peut échouer que par une exception du code appelé, et les observateurs vivent aussi longtemps qu'on les référence.

B. À travers le réseau. Lorsque sujet et observateurs vivent dans des JVM différentes — c'est le *callback* du CM 02 (diapositive 30) et la situation de l'étape A6 du TP 1 —, la structure du patron ne change pas, mais **chaque maillon change de nature** :

	OBSERVER LOCAL	OBSERVER À TRAVERS RMI
L'observateur	un objet ordinaire	un objet exporté par la JVM qui écoute
Ce que détient le sujet	une référence Java	un stub de l'observateur
Le contrat	une interface Java	une interface qui étend <i>Remote</i> , connue des deux côtés
La notification	un appel local	un appel distant, qui traverse une frontière de JVM
Un échec	une exception du code appelé	aussi une <i>RemoteException</i> : l'abonné a pu disparaître
Le fil d'exécution	celui qui publie	la notification arrive sur un fil du runtime RMI, chez l'abonné
Les rôles	sujet et observateurs	« client » et « serveur » s'inversent pendant le rappel

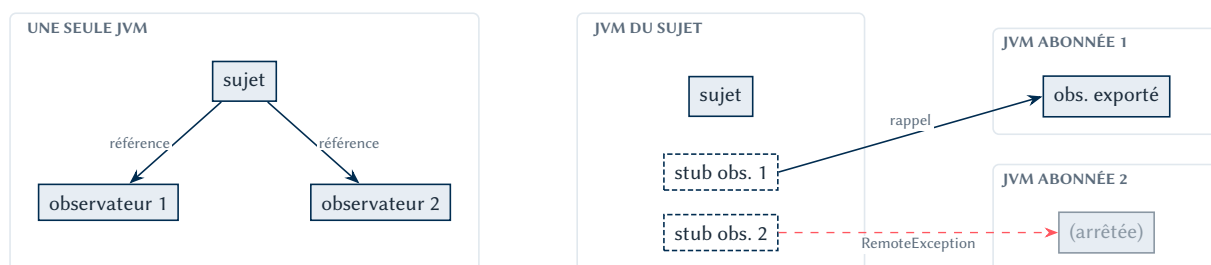


FIGURE 9 – Le même patron, local puis réparti

CE QUE CET EXEMPLE NE FAIT PAS

CapteurTemperature isole une *RuntimeException* d'observateur ; il ne traite aucune *RemoteException*, n'exporte rien et ne passe aucun stub. La partie 3 illustre le *principe* — un abonné défaillant ne doit pas priver les autres — que la version répartie devra appliquer à ses propres échecs. Le passage au réseau, avec son contrat distant, son exportation et ses cas de panne, est le travail de l'étape A6.

Dans l'UE. CM 01 §4, patron 5 · CM 02 §5, diapositive 30 (rappel du client par le serveur) · TP 1, étape A6.

6.8. Pièges et confusions

- « **Un Observer local suffit pour A6** ». Non : une référence Java ne traverse pas une frontière de JVM. Sans exportation, le sujet recevrait une *copie* sérialisée de l'observateur — ou un échec, si la classe n'est pas sérialisable —, et notifierait cette copie dans sa propre JVM.
- **Modifier la liste pendant qu'on la parcourt.** Avec une *ArrayList*, un désabonnement pendant la notification lève une *ConcurrentModificationException*.
- **Oublier le désabonnement.** Un abonné qui disparaît sans se désabonner reste dans la liste. En local, cela empêche le ramasse-miettes de le libérer ; à distance, chaque notification vers lui échoue.
- **Une notification lente bloque le sujet.** Les observateurs sont appelés l'un après l'autre : un abonné lent — ou injoignable, qui attend l'expiration d'un délai réseau — retarde tous les suivants.

CODE SOURCE

```
resources/design-patterns/observer — java -cp out patterns.observer.DemoObserver
```

7. Synthèse : les cinq patrons sur le chemin d'un appel distant

Pris isolément, chaque patron résout un problème de conception. Réunis, ils décrivent l'architecture d'un intergiciel à objets. Le schéma suivant place chacun sur le trajet d'un appel entre deux JVM ; les étiquettes en italique indiquent les patrons qui ne sont pas fournis par Java RMI lui-même mais apparaissent dans les technologies étudiées ensuite.

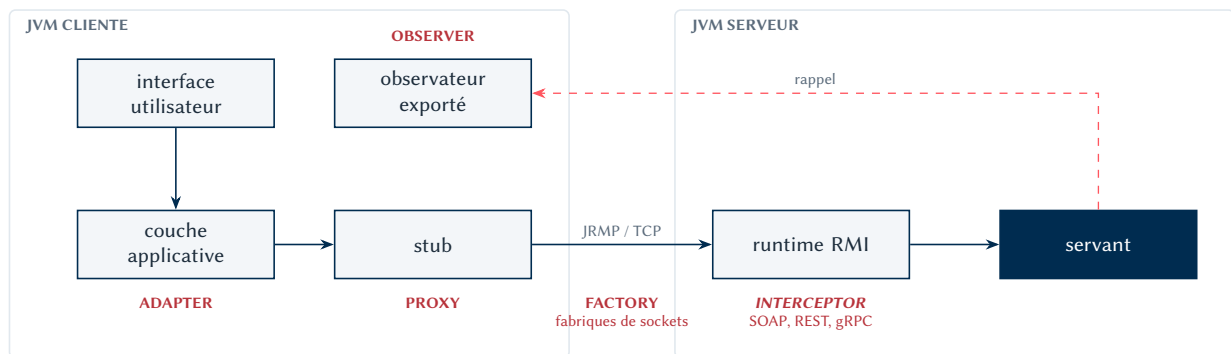


FIGURE 10 – Où chaque patron intervient lors d'un appel distant

PATRON	IDÉE À RETENIR	À RETROUVER
Proxy	un représentant de même interface contrôle l'accès	le stub RMI, point de contrôle A du TP 1 ; le client gRPC
Adapter	traduire un contrat en un autre, en un seul endroit	couche client séparée de l'interface utilisateur ; passerelles des TP 2 et TP 3
Factory	décider de la création ailleurs que là où l'on utilise	fabriques de sockets RMI (<i>java.rmi.server</i>)
Interceptor	encadrer l'appel sans toucher au métier	<i>handlers</i> SOAP, filtres REST, intercepteurs gRPC
Observer	notifier sans dépendre des abonnés	rappel du client par le serveur, étape A6 du TP 1